

Inference Rules

Concepts of Programming Languages
Lecture 5

Practice Problem

Determine an expression with the following type

$$\overbrace{('a \rightarrow 'a \rightarrow 'b)}^f \rightarrow \overbrace{('c \rightarrow 'a)}^g \rightarrow \overbrace{'c}^x \rightarrow 'b$$

$$\left(\begin{array}{l} \text{fun } f \rightarrow \text{fun } g \rightarrow \text{fun } x \rightarrow \\ \text{let } \overline{y} = g \ x \text{ in} \\ f \ y \ y \end{array} \right)$$

Outline

- » **Most important lecture!**
- » Discuss Formal Typing/Semantic Rules
- » Look at example rules for the constructs we've seen so far
- » Learn to read inference rules, i.e., translate mathematical notation to English and English to mathematical notation

Example

```
let x = 2 in x + x
```

- » Is this syntactically valid? Give me a proof
- » What is the type? Give me a proof
- » What is the value? Give me a proof
- » **Today:** Build a formal framework through which we develop these proofs

Recap

Recall: If-Expressions (Informal)

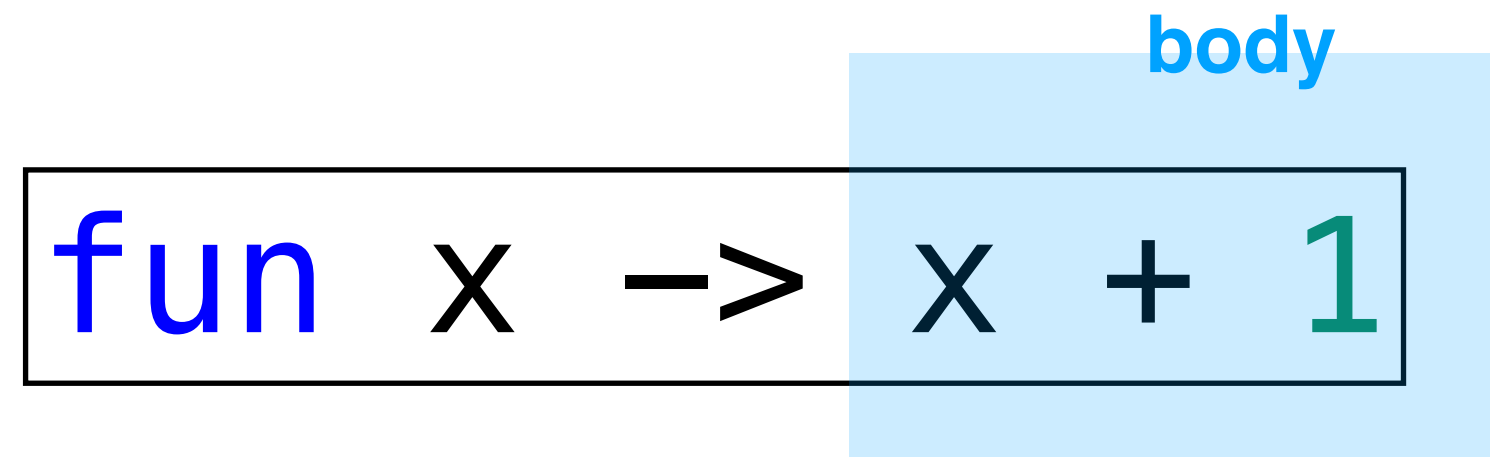
```
if x > 0 then x else -x
```

Syntax: if CONDITION then TRUE-CASE else FALSE-CASE

Typing: CONDITION must be a Boolean and TRUE-CASE and FALSE-CASE must be the same type. The type is then the same as that of TRUE-CASE and FALSE-CASE

Semantics: If CONDITION holds, then we get the TRUE-CASE, otherwise we get the FALSE-CASE

Recall: Functions (Informal)



body

```
fun x -> x + 1
```

Syntax: `fun VAR-NAME -> EXPR`

Typing: the type of a function is $T1 \rightarrow T2$ where $T1$ is the type of the input and $T2$ is the type of the output

Semantics: A function will evaluate to special *function value* (printed as `<fun>` by `utop`)

Recall: Application (Informally)

```
( fun x -> fun y -> x + y ) 3
```

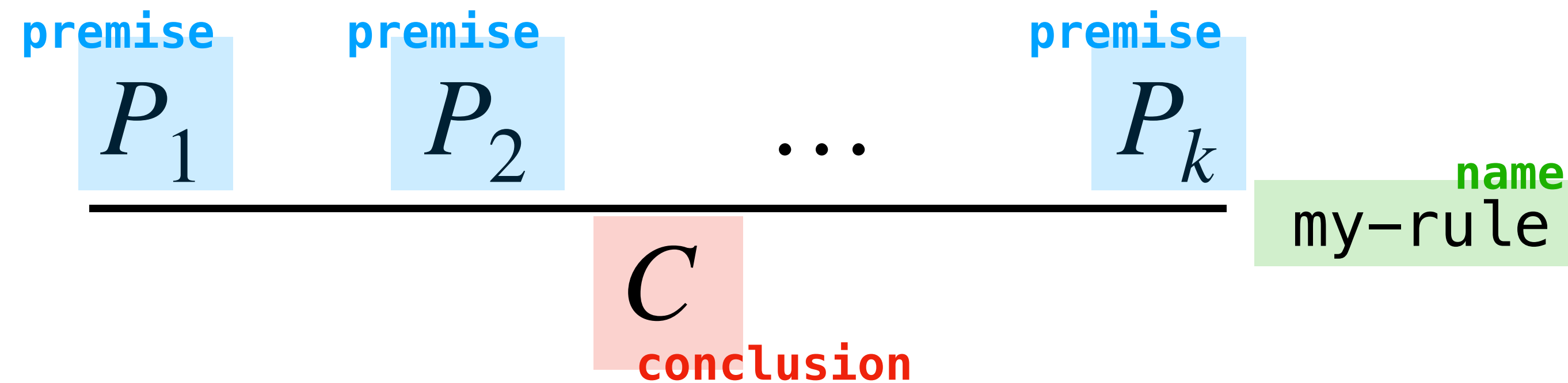
Syntax: FUNCTION-EXPR ARG-EXPR

Typing: If FUNCTION-EXPR is of type $T1 \rightarrow T2$,
and ARG-EXPR is of type $T1$, then the type is $T2$

Semantics: Substitute the value of ARG-EXPR into
the body of FUNCTION-EXPR and evaluate that

Inference Rules

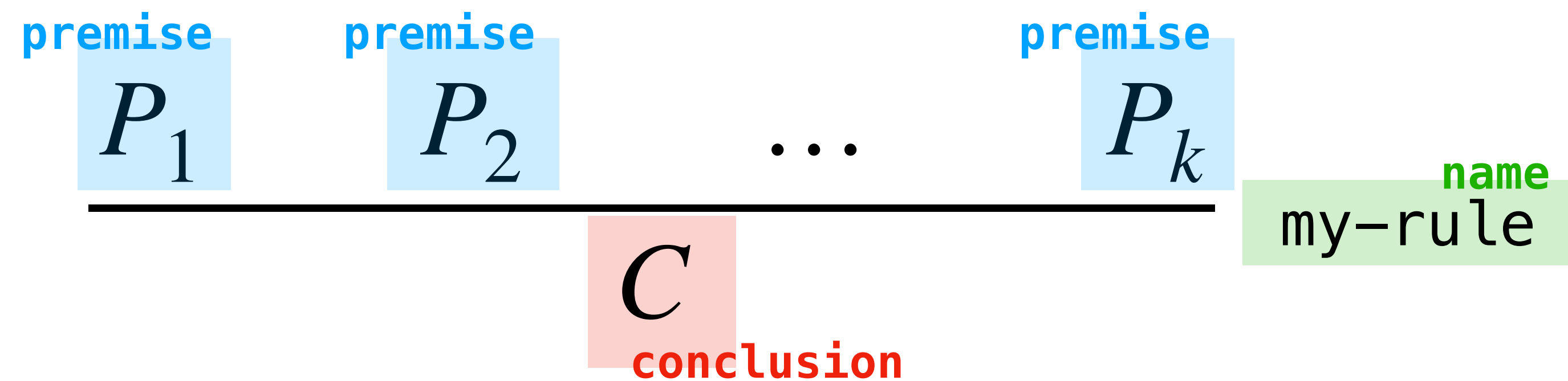
Inference Rules



Then general form of an inference rule has a collection of **premises** and a **conclusion**

There may be no premises: this is called an **axiom**

Inference Rules



We can read this as:

*If P_1 through P_k hold, then C holds (by **my-rule**)*

Judgements

- » Syntax judgments
- » Typing judgments
- » Semantic judgments

Judgements

- » Syntax judgments
- » Typing judgments
- » Semantic judgments

Syntax Judgements

$$e \in WF$$

A syntax judgement expresses that:

e is a well-formed expression

IMPORTANT: We will never use this kind of judgments explicitly!

Production Rules

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{expr} \rangle$

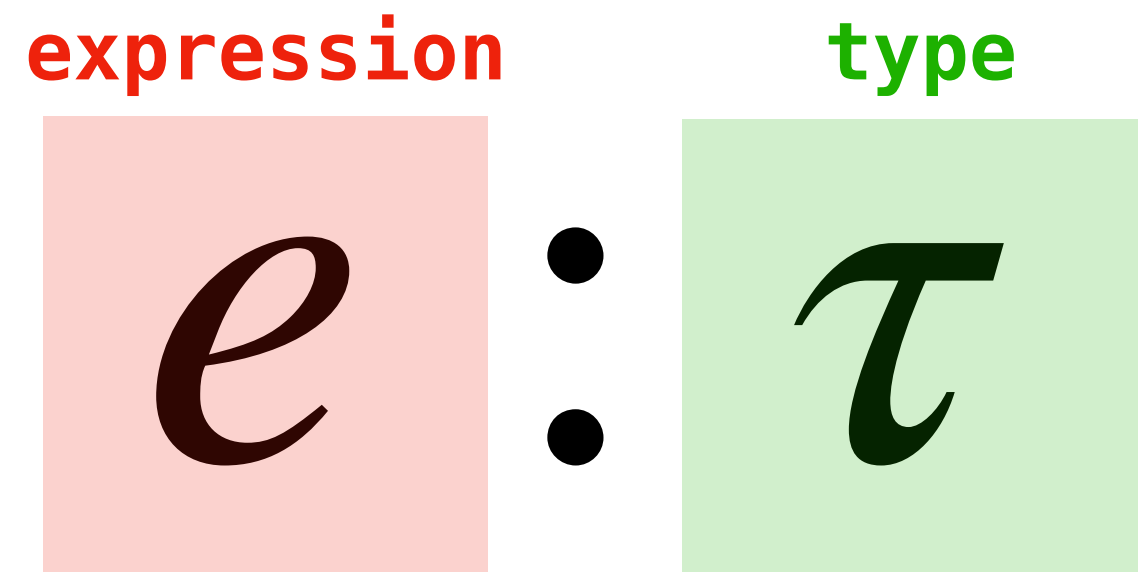
Instead it's standard to use **production rules**

(We'll spend more time on this when we cover formal grammar)

Judgements

- » Syntax judgments
- » **Typing judgments**
- » Semantic judgments

Typing Judgment and Rule



A typing judgment is a formal way of representing the statement:

e has type τ

A typing rule is an inference rule whose premises and conclusion are typing judgments

Typing Rule for Addition

We will use inference rules both to express typing rules and semantics rules

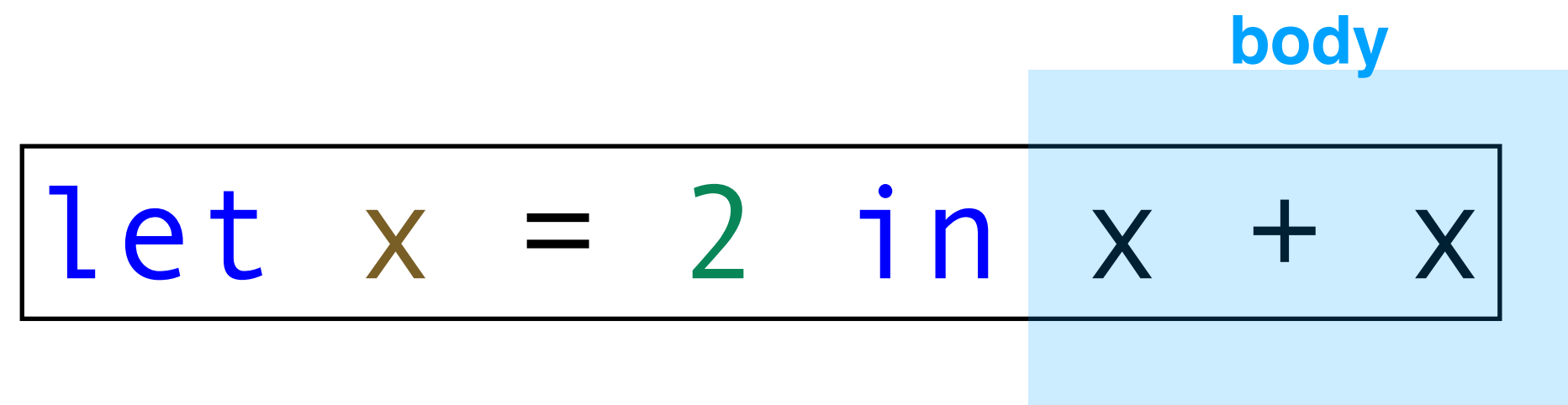
$$\frac{e_1 : \text{int} \quad e_2 : \text{int}}{e_1 + e_2 : \text{int}} \text{ (addInt)}$$

Say it in English!

*If (e_1 has type **int**) holds and (e_2 has type **int**) holds, then ($e_1 + e_2$ has type **int**) holds (by **addInt**)*

But this breaks down as soon
as we encounter variables

Let Expressions (Informal)



The diagram shows the code `let x = 2 in x + x` with syntax highlighting: `let` is blue, `x` is brown, `=` is black, `2` is green, `in` is blue, and `x + x` is blue. A light blue rectangular box labeled "body" in blue text at the top right encloses the expression `x + x`.

syntax: `let VARIABLE = EXPRESSION in BODY`

typing: *compute type of* EXPRESSION; *assume*
VARIABLE *has that type; compute type of* BODY

semantics: *compute value of* EXPRESSION; *substitute*
that value for VARIABLE *in* BODY

Let's Try to Write the Typing Rule

We will use inference rules both to express typing rules and semantics rules

$$\frac{e_1 : \tau_1 \quad e_2 : \tau_2}{\text{let } x = e_1 \text{ in } e_2 : \tau_2} \text{ (let)}$$

assuming x has type τ_1 ?

Say it in English!

*If $(e_1 \text{ has type } \tau_1)$ holds and $(e_2 \text{ has type } \tau_2)$ holds, then $(\text{let } x = e_1 \text{ in } e_2 \text{ has type } \tau_2)$ holds (by **let**)*

Example

`let x = 2 in x`

$$\frac{2 : \text{int} \quad x : \tau_2 \text{ What is } \tau_2?}{\text{let } x = 2 \text{ in } x : \tau_2} \text{ (let)}$$

We need to carry over the information that x has type `int` to the second premise. How do we do that?

Contexts

$$\Gamma = \{ x : \text{int}, y : \text{string}, z : \text{int} \rightarrow \text{string} \}$$

A **context** is a set of **variable declarations**

A variable declaration $(x : \tau)$ says: "I declare that the variable x is of type τ "

A context keeps track of all the types of variables in the "(static) environment" (or scope)

Typing Judgments

$$\text{context } \Gamma \vdash \text{expression } e : \text{type } \tau$$

A typing judgment is a compact way of representing the statement:

e is of type τ in the context Γ

A **typing rule** is an inference rule whose premises and conclusion are typing judgments

Corrected Integer Addition Typing Rule

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ (addInt)}$$

*If e_1 is an **int** (in any context Γ) and e_2 is an **int** then (in the same context Γ) $e_1 + e_2$ is an **int** (in the same context Γ)*

Example: Reading Typing Judgements

$$\{b : \text{bool}\} \vdash \text{if } b \text{ then } 2 \text{ else } 3 : \text{int}$$

In English: *Given that b is a bool , the expression $\text{if } b \text{ then } 2 \text{ else } 3$ is an int*

The context allows us to determine the type of an expression *relative to the types of variables*

$$\{f : \text{int} \rightarrow \text{string}\} \vdash f \ 2 : \text{string}$$
$$\{f : \text{int} \rightarrow \text{bool}\} \vdash f \ 2 : \text{bool}$$

Judgements are claims, not truths

$\{b : \text{bool}\} \vdash \text{if } b \text{ then } 2 \text{ else } 3 : \text{string}$

A judgement is a *claim* in the same way that "there are infinitely many twin primes" or "pigs fly" is a claim

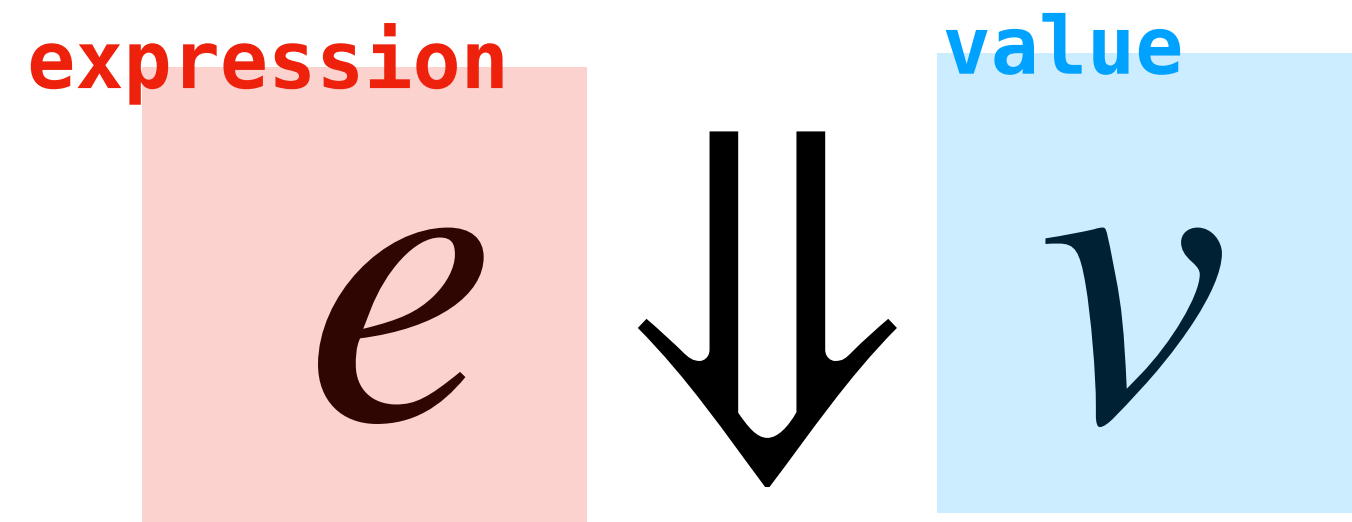
We haven't **proved** anything by writing down a typing judgment

Next time: we'll talk about **typing derivations**, which are used to demonstrate that expressions *actually* have their desired types in our PL

Judgements

- » Syntax judgments
- » Typing judgments
- » **Semantic judgments**

Semantic Judgements



A semantic judgement is a compact way of representing the statement:

The expression e evaluates to the value v

A **semantic rule** is an inference rule with semantic judgements

Recall: Integer Addition Semantic Rule

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2}{e_1 + e_2 \Downarrow v_1 + v_2} \text{ (evalInt)}$$

If e_1 evaluates to the (integer) v_1 and e_2 evaluates to the (integer) v_2 , then $e_1 + e_2$ evaluates to the (integer) $v_1 + v_2$

Example: Reading Semantic Judgments

if 2 > 3 then 2 + 2 else 3 $\Downarrow$ 3

In English: The expression

if 2 > 3 then 2 + 2 else 3

evaluates to the value 3

320Caml Inference Rules

Reminder: for every expression in our language, we given inference rules for syntax, typing, and semantics

Expressions

» Let-expressions

» If-Expressions

» Functions

» Application

Expressions

» **Let-expressions**

» If-Expressions

» Functions

» Application

Let-Expressions (Syntax Rule)

$\langle \text{expr} \rangle ::= \text{let } \langle \text{var} \rangle = \langle \text{expr} \rangle \text{ in } \langle \text{expr} \rangle$

If x is a valid variable name, and e_1 is a well-formed expression and e_2 is a well-formed expression then

$\text{let } x = e_1 \text{ in } e_2$

is a well-formed expression

Let-Expressions (Syntax Rule)

$\langle \text{expr} \rangle ::= \text{let } \langle \text{var} \rangle = \langle \text{expr} \rangle \text{ in } \langle \text{expr} \rangle$

If x is a valid variable name, and e_1 is a well-formed expression and e_2 is a well-formed expression then

$\text{let } x = e_1 \text{ in } e_2$

is a well-formed expression

Let-Expressions (Typing Rule)

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau} \text{ (let)}$$

If e_1 is of type τ_1 in the context Γ , and e_2 is of type τ in the context Γ *with the variable declaration* $(x : \tau_1)$ *added to it*, then

let $x = e_1$ **in** e_2

is of type τ in the context Γ

Let-Expressions (Typing Rule)

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau} \text{ (let)}$$

Note: Look at how much more compact the rule is!

If e_1 is of type τ_1 in the context Γ , and e_2 is of type τ in the context Γ *with the variable declaration* $(x : \tau_1)$ *added to it*, then

$\text{let } x = e_1 \text{ in } e_2$

is of type τ in the context Γ

Let-Expressions (Semantic Rule)

$$\frac{e_1 \Downarrow v_1 \quad [v_1 / x] e_2 \Downarrow v}{\text{let } x = e_1 \text{ in } e_2 \Downarrow v}$$

If e_1 evaluates to v_1 and e_2 with v_1 substituted for x evaluates to v , then

let $x = e_1$ **in** e_2

evaluates to v

Let-Expressions (Semantic Rule)

$$\frac{e_1 \Downarrow v_1 \quad [v_1/x]e_2 \Downarrow v}{\text{let } x = e_1 \text{ in } e_2 \Downarrow v} \text{ (letEval)}$$

If e_1 evaluates to v_1 and e_2 with v_1 substituted for x evaluates to v , then

$\text{let } x = e_1 \text{ in } e_2$

evaluates to v

Expressions

» Let-expressions

» **If-Expressions**

» Functions

» Application

If-Expressions (Syntax Rule)

$\langle \text{expr} \rangle ::= \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{expr} \rangle \text{ else } \langle \text{expr} \rangle$

If-Expressions (Syntax Rule)

$\langle \text{expr} \rangle ::= \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{expr} \rangle \text{ else } \langle \text{expr} \rangle$

If e_1 is a well-formed expression and e_2 is a well-formed expression and e_3 is a well-formed expression, then

$\text{if } e_1 \text{ then } e_2 \text{ else } e_3$

is a well-formed expression

If-Expressions (Typing Rule)

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau}$$

If-Expressions (Typing Rule)

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{(if)}$$

If e_1 is of type `bool` in the context Γ and e_2 and e_3 are of type τ in the context Γ , then

`if  $e_1$  then  $e_2$  else  $e_3$`

is of type τ in the context Γ

If-Expressions (Semantics)

$$\frac{e_1 \Downarrow \text{true} \qquad e_2 \Downarrow v}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v}$$

If-Expressions (Semantic Rule 1)

$$\frac{e_1 \Downarrow \text{true} \quad e_2 \Downarrow v_2}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_2} \text{ (ifEvalTrue)}$$

If e_1 evaluates to T and e_2 evaluates to v_2 , then

$\text{if } e_1 \text{ then } e_2 \text{ else } e_3$

evaluates to v_2

If-Expressions (Semantic Rule 2)

$$\frac{e_1 \Downarrow \text{false} \quad e_3 \Downarrow v_3}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_3} \text{ (ifEvalFalse)}$$

Note: we never evaluate both branches

If e_1 evaluates to $\perp$ and e_2 evaluates to v_2 , then

if e_1 then e_2 else e_3

evaluates to v_3

Expressions

» Let-expressions

» If-Expressions

» **Functions**

» Application

Functions (Syntax Rule)

$\langle \text{expr} \rangle ::= \text{fun } \langle \text{var} \rangle \rightarrow \langle \text{expr} \rangle$

Functions (Syntax Rule)

$$\langle \text{expr} \rangle ::= \text{fun } \langle \text{var} \rangle \rightarrow \langle \text{expr} \rangle$$

If x is a valid variable name and e is a well-formed expression, then

$$\text{fun } x \rightarrow e$$

is a well-formed expression

Functions (Typing Rule)

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2}$$

Functions (Typing Rule)

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2} \text{ (fun)}$$

If e has type τ_2 in the context Γ with the variable declaration $(x : \tau_1)$ added, then

$\text{fun } x \rightarrow e$

is of type $\tau_1 \rightarrow \tau_2$ in the context Γ

Functions (Semantic Rule)

$\text{fun } x \rightarrow e \Downarrow \text{fun } x \rightarrow e$

Functions (Semantic Rule)

$$\frac{}{\text{fun } x \rightarrow e \Downarrow \lambda x . e} \text{ (funEval)}$$

Under no premises, the expression

$\text{fun } x \rightarrow e$

evaluates to the *function value* $\lambda x . e$ (we'll talk more about function values later)

Expressions

» Let-expressions

» If-Expressions

» Functions

» **Application**

Application (Syntax Rule)

Application (Syntax Rule)

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle \langle \text{expr} \rangle$

If e_1 is a well-formed expression and e_2 is a well-formed expression, then $e_1 e_2$ is a well-formed expression

Application (Typing Rule)

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau}$$

If e_1 has type $\tau_2 \rightarrow \tau$ under the context Γ and e_2 is of type τ_2 under the context Γ , then $e_1 e_2$ is of type τ under the context Γ

Application (Typing Rule)

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau} \text{ (app)}$$

If e_1 has type $\tau_2 \rightarrow \tau$ under the context Γ and e_2 is of type τ_2 under the context Γ , then $e_1 e_2$ is of type τ under the context Γ

Application (Semantic Rule)

$$\frac{e_1 \Downarrow \lambda x . e \quad e_2 \Downarrow v_2 \quad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v} \text{(appEval)}$$

1. e_1 evaluates to a function value $\lambda x . e$
2. e_2 evaluates to v_2
3. e with v_2 substituted for x evaluates to v

It follows that $e_1 e_2$ evaluates to v

We 'll see more typing
rules and semantic rules

We'll also give a written
reference for the rules we talk
about in class

Summary

Inference rules formally describe how the typing and semantics of a programming language work